Introduction to Deep Learning

Artificial Neural Network, Recurrent Neural Network

Created By : Abdan Hafidz

Abstract

This module serves as a comprehensive guide to the mathematical and theoretical underpinnings of Neural Networks. We journey from the atomic unit of the Single Layer Perceptron, through the complexity of Multi-Layer Perceptrons and Activation Functions, to the temporal dynamics of Recurrent Neural Networks. The focus is on intuitive understanding derived from rigorous mathematical abstraction.

Contents

1 Introduction: The Mathematical Brain

Deep Learning is often mystified as a ”black box” of magic, but at its core, it is nothing more than refined linear algebra and calculus seeking to minimize an error. Imagine a child learning to distinguish between a hot stove and a toy. The child receives input (visuals, heat), processes it, makes a decision (touch or don't touch), and receives feedback (pain or fun). Over time, the internal connections in the child's brain adjust to minimize the ”error” (pain).

Artificial Neural Networks mimic this exact process. We are essentially trying to map a set of inputs to a set of outputs by passing them through a series of mathematical filters. These filters are adjustable. We tweak them—value by value—until the system behaves the way we want.

2 The Single Layer Perceptron (SLP)

The story begins with the Perceptron, proposed by Frank Rosenblatt in 1958. It is the atom of deep learning. While biological neurons are complex organic switches, the Perceptron is a simplified mathematical abstraction of a biological neuron.

2.1 Theoretical Abstraction

A Perceptron takes a vector of input numbers, weighs them according to their importance, sums them up, adds a bias (a threshold), and then squashes the result to make a decision.

Let us define our input vector $\mathbf{x} = [x_1, x_2, ..., x_n]$. These inputs could be pixel brightness values or financial indicators. Associated with each input is a weight w_i, representing the strength of the connection. We also have a bias term b, which allows the neuron to activate even when all inputs are zero.

The weighted sum, denoted as z, is calculated as follows:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

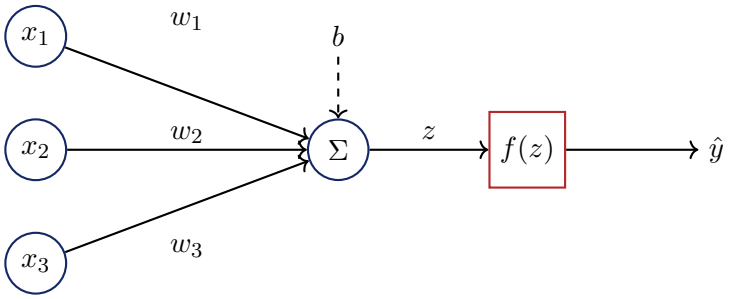
In vector notation, this becomes an elegant dot product:

$$z = \mathbf{w}^T \mathbf{x} + b$$

Once we have z, we need to decide if the neuron ”fires” or not. In the earliest versions, this was a step function. If $z > 0$, output 1; otherwise, output 0.

2.2 Visualizing the Perceptron

The following diagram illustrates the information flow within a single neuron. Inputs flow from left to right, are scaled by weights, summed, and processed.



3 Activation Functions: The Spark of Nonlinearity

If we only used the weighted sum z, our network would be nothing more than a linear regression model. No matter how many layers we stack, a sum of linear functions is still just a linear function. To learn complex patterns—like the curve of a handwritten number '8'—we need nonlinearity. This is the role of the Activation Function.

3.1 Sigmoid Function

Historically popular, the Sigmoid squashes any number into a range between 0 and 1. It is interpreted as a probability.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

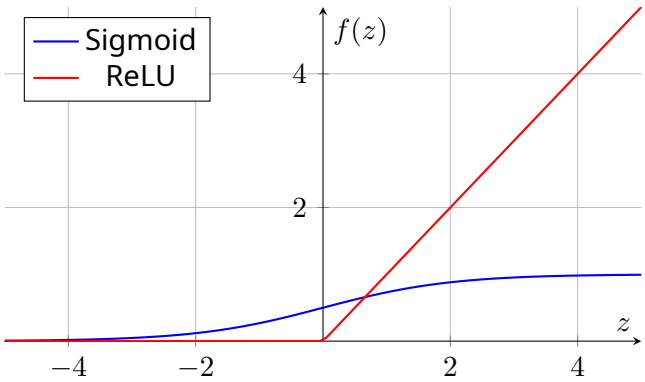
However, it suffers from the "vanishing gradient" problem, where the derivative becomes incredibly small at the extremes, causing learning to stall.

3.2 ReLU (Rectified Linear Unit)

The modern default. It is deceptively simple: if the input is positive, pass it through; if negative, output zero.

$$f(z) = \max(0, z)$$

Despite its simplicity, ReLU allows networks to train much faster because its derivative is either 0 or 1, preventing the gradient from vanishing during the training of deep networks.



4 The Multi-Layer Perceptron (MLP)

A single perceptron can only solve linearly separable problems (it cannot solve XOR). To solve complex problems, we arrange neurons in layers.

4.1 Architecture

An MLP consists of an **Input Layer**, one or more **Hidden Layers**, and an **Output Layer**. The "Hidden" layers are where the feature extraction occurs.

The mathematics expands to matrices. If layer l has n neurons and layer $l-1$ has m neurons, the weights connecting them form a matrix $W^{[l]}$ of size $n \times m$.

The forward propagation step for layer l is:

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$$

$$\mathbf{a}^{[l]} = g(\mathbf{z}^{[l]})$$

where $\mathbf{a}^{[0]} = \mathbf{x}$ (the input data) and g is the activation function.

4.2 Learning: Backpropagation

This is the heart of deep learning. We define a Cost Function J (e.g., Mean Squared Error) that measures how wrong the network is.

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{2}||\hat{y} - y||^2$$

To improve the network, we must adjust the weights to lower J. We use Gradient Descent. We calculate the gradient of the cost with respect to the weights using the **Chain Rule** of calculus, moving backward from the output to the input.

$$w_{new} = w_{old} - \alpha \frac{\partial J}{\partial w}$$

Here, α is the learning rate. We are literally walking down the hill of error to find the valley of correctness.

5 Recurrent Neural Networks (RNN)

Standard MLPs assume that inputs are independent of each other. But what if we are processing a sentence? The meaning of a word depends on the words that came before it. MLPs have no memory; RNNs do.

5.1 The Hidden State

In an RNN, the decision at time step t depends on the current input x_t AND the previous hidden state h_{t-1}. The hidden state acts as the ”memory” of the network.

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

Here, W_{hh} connects the previous memory to the current memory, and W_{xh} connects the new input to the memory.

5.2 Visualizing the Loop

An RNN can be visualized as a loop that passes information to the next step, or ”unrolled” over time.

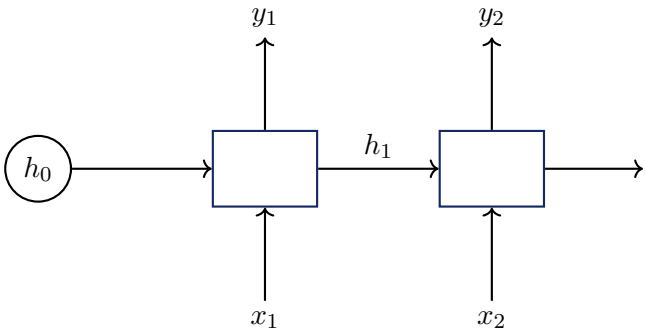


Figure: An RNN unrolled over two time steps.

6 Case Study: The ”Exam Pass” Prediction

Let us solidify these concepts with a manual calculation. We want to predict if a student passes ($y = 1$) or fails ($y = 0$) based on Study Hours (x_1) and Sleep Hours (x_2).

Dataset:

Study (x_1)	**Sleep (x_2)**	**Pass (y)**
2	5	1
8	7	1
1	2	0
3	8	0

For this manual calculation, we will process the **first sample** (Study: 2, Sleep: 5, Pass: 1).

Architecture: A single neuron (Perceptron) with Sigmoid activation.
Initialization: Weights $w_1 = 0.5$, $w_2 = -0.1$, Bias $b = 0$.
Learning Rate: $\alpha = 0.1$.

Step 1: Forward Propagation

First, we calculate the weighted sum (the logit):

$$z = w_1 x_1 + w_2 x_2 + b$$

$$z = (0.5 \times 2) + (-0.1 \times 5) + 0$$

$$z = 1.0 - 0.5 = 0.5$$

Next, we apply the Sigmoid activation function to get the prediction $\hat{y}$:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-0.5}}$$

$$\hat{y} \approx \frac{1}{1 + 0.606} \approx 0.622$$

The network predicts a 62.2% chance of passing.

Step 2: Calculate Error

The true value y is 1. The error (Loss) using Mean Squared Error is:

$$J = \frac{1}{2}(y - \hat{y})^2 = \frac{1}{2}(1 - 0.622)^2 = \frac{1}{2}(0.378)^2 \approx 0.071$$

Step 3: Backpropagation (Gradients)

We need to know how to change w_1 to reduce the error. We use the Chain Rule:

$$\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$

1. Change in Error w.r.t Prediction: $\frac{\partial J}{\partial \hat{y}} = -(y - \hat{y}) = -(1 - 0.622) = -0.378$
2. Change in Prediction w.r.t z (Sigmoid derivative): $\sigma'(z) = \hat{y}(1 - \hat{y}) = 0.622(1 - 0.622) \approx 0.235$
3. Change in z w.r.t Weight w_1: $\frac{\partial z}{\partial w_1} = x_1 = 2$

Combining them:

$$\frac{\partial J}{\partial w_1} = (-0.378) \times (0.235) \times (2) \approx -0.177$$

Step 4: Update Weights

We update the weight by subtracting the gradient multiplied by the learning rate:

$$w_{1,new} = w_1 - \alpha \frac{\partial J}{\partial w_1}$$

$$w_{1,new} = 0.5 - 0.1(-0.177) = 0.5 + 0.0177 = 0.5177$$

Notice that w_1 increased. This makes sense! Since study hours (x_1) are positive and we wanted a higher output (closer to 1), the network learned to increase the importance of study hours. This is the fundamental mechanism of machine learning.

7 Conclusion

We have traversed the landscape of Deep Learning from the static mathematics of the Perceptron to the dynamic memory of Recurrent Neural Networks. While the architectures grow more complex—spanning Transformers and Convolutional networks—the underlying logic remains the same: a symphony of dot products, non-linearities, and the relentless pursuit of lower error through calculus.